Deep Reinforcement Learning: Policy Gradient

Yuanchu Liang Jingyang You

The Australian National University

yuanchu.liang@anu.edu.au jingyang.you@anu.edu.au

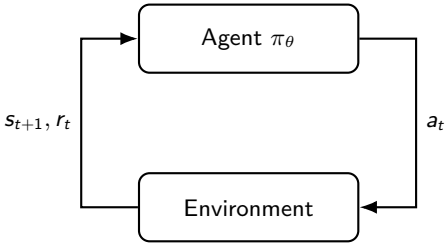
Why Reinforcement Learning?

Core setting: an agent repeatedly acts, observes consequences, and improves its behaviour from scalar feedback.

- **Robotics:** locomotion, manipulation, navigation.
- **Games:** Go, Atari, Mahjong, real-time strategy.
- **Sequential decisions:** recommendation, dialogue, resource allocation.

Distinctive difficulty:

- actions affect future data distribution;
- rewards may be delayed and sparse;
- exploration and exploitation are coupled.



Learning problem

Choose a policy that maximises long-term reward, not just immediate prediction accuracy.

Markov Decision Processes: Formal Model

A discounted Markov decision process is

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \rho_0, \gamma), \qquad \gamma \in [0, 1).$$

- $s_0 \sim \rho_0$, $a_t \sim \pi_\theta(\cdot \mid s_t)$, $s_{t+1} \sim P(\cdot \mid s_t, a_t)$.
- $r(s_t, a_t)$ is the one-step reward; stochastic rewards can be included by taking conditional expectation.
- The Markov property: future dynamics depend on history only through (s_t, a_t).

For a trajectory $\tau = (s_0, a_0, s_1, a_1, \ldots)$,

$$p_\theta(\tau) = \rho_0(s_0) \prod_{t \geq 0} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t).$$

Control objective

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right].$$

Value Functions and Bellman Equations

For a fixed policy π, define

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^{k} r(s_{t+k}, a_{t+k}) \mid s_t = s\right],$$

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^{k} r(s_{t+k}, a_{t+k}) \mid s_t = s, a_t = a\right].$$

Policy evaluation:

$$V^{\pi}(s) = \mathbb{E}_{a\sim\pi}\left[r(s,a) + \gamma \mathbb{E}_{s'\sim P} V^{\pi}(s')\right].$$

$$Q^{\pi}(s,a) = r(s,a) + \gamma \mathbb{E}_{s'\sim P, a'\sim\pi} Q^{\pi}(s',a').$$

Optimality:

$$V^{*}(s) = \max_{a}\left\{r(s,a) + \gamma \mathbb{E}_{s'\sim P} V^{*}(s')\right\}.$$

$$Q^{*}(s,a) = r(s,a) + \gamma \mathbb{E}_{s'\sim P} \max_{a'} Q^{*}(s',a').$$

Intuition

Bellman equations express a global return as immediate reward plus the value of the next decision point.

Policy Gradient

Policy gradient methods directly optimise a differentiable stochastic policy $\pi_\theta(a \mid s)$. Write the objective as an integral over trajectories,

$$J(\theta) = \int p_\theta(\tau) R(\tau) \, \mathrm{d}\tau, \qquad R(\tau) = \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t). \qquad p_\theta(\tau) = \rho_0(s_0) \prod_{t \geq 0} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t).$$

Gradient-based Optimisation. It is natural to consider the gradient of the above formula

$$\nabla_\theta J(\theta) = \int \nabla_\theta p_\theta(\tau) \, R(\tau) \, \mathrm{d}\tau$$

However, the integrand cannot be computed/approximated directly. The log-trick makes it more useful

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau). \qquad \nabla_\theta J(\theta) = \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) \, R(\tau) \, \mathrm{d}\tau = \mathbb{E}_{\tau \sim p_\theta} \left[\nabla_\theta \log p_\theta(\tau) \, R(\tau) \right].$$

The gradient has become an expectation that can be Monte-Carlo approximated. Let's further simplify this term.

Policy Gradient

Now we are at

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\nabla_\theta \log p_\theta(\tau)\, R(\tau) \right].$$

Note that

$$\log p_\theta(\tau) = \log \rho_0(s_0) + \sum_{t \geq 0} \Big[\log \pi_\theta(a_t \mid s_t) + \log P(s_{t+1} \mid s_t, a_t) \Big].$$

Only the policy terms depend on θ; the gradients of $\log \rho_0$ and $\log P$ vanish:

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t \geq 0} \nabla_\theta \log \pi_\theta(a_t \mid s_t).$$

Further simplifying on the indices

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t \geq 0} \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t \right], \qquad G_t = \sum_{k \geq t} \gamma^{k-t} r_k.$$

Policy Gradient Theorem

We are at

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t \geq 0} \gamma^t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t) \, G_t \right], \qquad G_t = \sum_{k \geq t} \gamma^{k-t} r_k.$$

As $\mathbb{E}[G_t \mid s_t, a_t] = Q^{\pi_\theta}(s_t, a_t)$

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t \geq 0} \gamma^t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t) \, Q^{\pi_\theta}(s_t, a_t) \right].$$

Policy Gradient Theorem

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t \geq 0} \gamma^t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t) \, Q^{\pi_\theta}(s_t, a_t) \right]$$

REINFORCE, Baselines, and Variance Reduction

REINFORCE proposes a naive Monte-Carlo estimate of $\nabla_\theta J(\theta)$, where

$$\widehat{g}_{\mathrm{RF}} = \frac{1}{N}\sum_{i=1}^{N}\sum_{t\geq 0}\gamma^t\,\nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)})\,\widehat{Q}_t^{(i)}, \text{ where } \widehat{Q}_t = \sum_{k=t}^{T-1}\gamma^{k-t} r(s_k, a_k).$$

However, the variance of such estimation is very high.

Policy Gradient with Baselines

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\left[\sum_{t\geq 0}\gamma^t\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)\,\left(Q^{\pi_\theta}(s_t, a_t) - b(s_t)\right)\right]$$

Baseline A state-dependent baseline does not change the expected gradient:

$$\mathbb{E}_{a_t\sim\pi_\theta(\cdot|s_t)}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) b(s_t)\right] = b(s_t)\nabla_\theta \sum_a \pi_\theta(a \mid s_t) = 0.$$

Advantage $b(s_t) = V^{\pi_\theta}(s_t)$ is a common choice for a reduced variance. $A^{\pi_\theta}(s_t, a_t) := Q^{\pi_\theta}(s_t, a_t) - V^{\pi_\theta}(s_t)$ is named Advantage.

Advantage Estimation in Practice

Now the problem becomes how to estimate $A_t^{\pi_\theta} = Q^{\pi_\theta}(s_t, a_t) - V^{\pi_\theta}(s_t)$.

Temporal Difference (TD). We note that

$$\begin{aligned} A_t^{\pi_\theta} &= r(s_t, a_t) + \gamma \sum_{s'} P(s'|s_t, a_t) V^{\pi_\theta}(s') - V^{\pi_\theta}(s_t) \\ &\approx r(s_t, a_t) + \gamma V^{\pi_\theta}(s_{t+1}) - V^{\pi_\theta}(s_t) := \widehat{A}_t^{(1)} \\ &\approx r(s_t, a_t) + \gamma\, r(s_{t+1}, a_{t+1}) + \gamma^2\, r(s_{t+2}, a_{t+2}) + \cdots + \gamma^n V^{\pi_\theta}(s_{t+n}) - V^{\pi_\theta}(s_t) := \widehat{A}_t^{(n)} \end{aligned}$$

We name $\widehat{A}_t^{(1)}$ a **one-step TD** estimate, $\widehat{A}_t^{(n)}$ a **n-step TD** estimate of $A_t^{\pi_\theta}$.

Generalized Advantage Estimation (GAE). GAE is the de facto method for estimating the advantage.

$$\widehat{A}_t^{\mathrm{GAE}} = (1-\lambda) \sum_{n \geq 1} \lambda^{n-1} \widehat{A}_t^{(n)}$$

Both methods require $V^{\pi_\theta}(\cdot)$, which is not directly accessible. In practice this is usually approximated using a neural network $V_\phi(\cdot)$.

Issues with Policy Updates

Suppose one has obtained $\nabla_\theta J(\theta)$ and would like to use it for policy updates, i.e.

$$\theta_{\text{new}} \leftarrow \theta_{\text{old}} + \alpha \cdot \nabla_\theta J(\theta)|_{\theta=\theta_{\text{old}}}$$

Such naive gradient update is only safe to perform once. **Why?**

Because the data (trajectories) are dependent on θ. An updated θ_{new} needs to be updated from gradients computed from new trajectories which are collected following $\pi_{\theta_{\text{new}}}$.

Is there a more controlled way to perform gradient update multiple times using the same batch of data?

PPO considers optimising instead a surrogate objective function that allows safer multiple gradient updates.

PPO Objective and Training Recipe

The clipped surrogate objective is

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\widehat{A}_t, \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\widehat{A}_t \right) \right].$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t|s_t)}$.

- If $\widehat{A}_t > 0$, PPO prevents excessively increasing the action probability.
- If $\widehat{A}_t < 0$, PPO prevents excessively decreasing the action probability.
- The minimum makes the objective pessimistic relative to the unclipped surrogate.
- Clipping is a per-sample, first-order proxy for the KL trust region: once $r_t(\theta)$ leaves $[1-\epsilon, 1+\epsilon]$ in the beneficial direction, the gradient incentive vanishes.

A common implementation optimises

$$\max_{\theta,\phi} \quad L^{\mathrm{CLIP}}(\theta) - c_v \mathbb{E}_t \left[\left(V_\phi(s_t) - \widehat{G}_t \right)^2 \right] + c_H \mathbb{E}_t \left[\mathcal{H}(\pi_\theta(\cdot \mid s_t)) \right].$$

Typical workflow

Collect trajectories on-policy, compute $\widehat{A}_t$, run several minibatch epochs, then discard the data.

Putting It Together: Practical Takeaways

Algorithmic progression

- **Policy gradient:** unbiased direction, often high variance.
- **PPO:** stable on-policy updates with a clipped surrogate.

Design questions

- Is the policy stochastic or deterministic?
- Is the data on-policy or off-policy?
- How is the advantage or Bellman target estimated?
- What stabilisation is used: clipping, targets, double critics, entropy?

Main message

Modern deep RL is best viewed as approximate dynamic programming plus stochastic optimisation under a changing data distribution.